

Full Length Article

Meta-path and context-aware learning for attribute completion in heterogeneous graphs

Geng Chen^{1,*}, Yuan Feng¹, Lijun Zhang¹, Xiaoyu Bai, Qingyue Wang, Peng Wang^{*}

National Engineering Laboratory for Integrated Aero-Space-Ground-Ocean Big Data Application Technology, School of Computer Science and Engineering, Northwestern Polytechnical University, 710072, Xi'an, China

ARTICLE INFO

Keywords:

Graph neural networks
Attribute completion
Heterogeneous graph learning
Graph node classification

ABSTRACT

Heterogeneous graph attribute completion (HGAC) has garnered substantial attention due to its critical role in various graph-based applications in recent years. However, existing HGAC methods often overlook valuable meta-path information, focus solely on neighboring nodes during attribute completion, and exhibit inefficiencies when integrated with heterogeneous graph neural networks for downstream tasks. To address these challenges, we propose EMC-Net, a novel learning framework that incorporates meta-path information and context-aware learning. Our framework employs collaborative meta-path-driven embedding schemes, which capture the rich semantic information embedded in heterogeneous graphs, and introduce context-aware attention mechanisms to dynamically adjust the importance of different nodes and edges during attribute completion. Our framework not only enhances the accuracy of attribute predictions but also improves the efficiency and scalability of the model. Extensive experimental results on benchmark datasets demonstrate that our framework significantly outperforms existing HGAC methods in terms of both accuracy and computational efficiency. This advancement offers new insights and methodologies for developing more robust and effective heterogeneous graph attribute completion technologies.

1. Introduction

Graphs are common data structures that have been widely used in various fields, such as social network analysis and recommendation systems. According to the number of node types and edge types, graphs can be divided into homogeneous graphs and heterogeneous graphs. As shown in Fig. 1, the graph structure data is incomplete in the IMDB dataset collected from the real world, with only a small fraction of samples containing complete attributes. This information is often not fully available in practical situations due to the unavailability of private information of many individuals in applications, such as social networks. There are generally two kinds of methods to handle missing data. Methods in the first category utilize conventional machine learning techniques to impute the absent data, subsequently leveraging the completed data for downstream applications. Typical representative methods include mean imputation (García-Laencina et al., 2010), matrix factorization (Koren et al., 2009), multiple imputation (Rubin, 2004), etc. Methods in the second category combine attribute completion and downstream tasks of heterogeneous graphs using end-to-end graph neu-

ral networks. These methods have shown promising results and have gained increasing popularity. Both supervised (Chen et al., 2020b; Jin et al., 2021; Taguchi et al., 2021) and unsupervised methods (He et al., 2022) based on graph neural networks have been proposed.

Despite their progress, existing methods suffer from three major limitations. Firstly, they have primarily utilized random walks for embedding learning and have not taken into account the significance of meta-paths. This approach leads to the loss of valuable prior knowledge due to the presence of diverse node types in heterogeneous graphs. Secondly, attribute completion in existing methods has been heavily reliant on information from neighboring nodes, leading to the exclusion of crucial information about non-neighbor nodes. Lastly, existing methods have employed a heterogeneous graph neural network for downstream tasks after attribute completion, which results in low computational efficiency.

To address these limitations, we propose an efficient meta-path context-aware learning framework (i.e., EMC-Net) for improved HGAC. Our EMC-Net employs a meta-path-driven random walk embedding approach that incorporates prior knowledge of meta-path while sampling

* Corresponding authors.

E-mail addresses: geng.chen@nwpu.edu.cn (G. Chen), 1634893188@qq.com (Y. Feng), lijunzhang@mail.nwpu.edu.cn (L. Zhang), 943338868@qq.com (X. Bai), wm33564@126.com (Q. Wang), peng.wang@nwpu.edu.cn (P. Wang).

¹ G. Chen, Y. Feng, and L. Zhang contributed equally to this work.

<https://doi.org/10.1016/j.neunet.2026.108895>

Received 11 September 2025; Received in revised form 14 January 2026; Accepted 23 March 2026

Available online 25 March 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

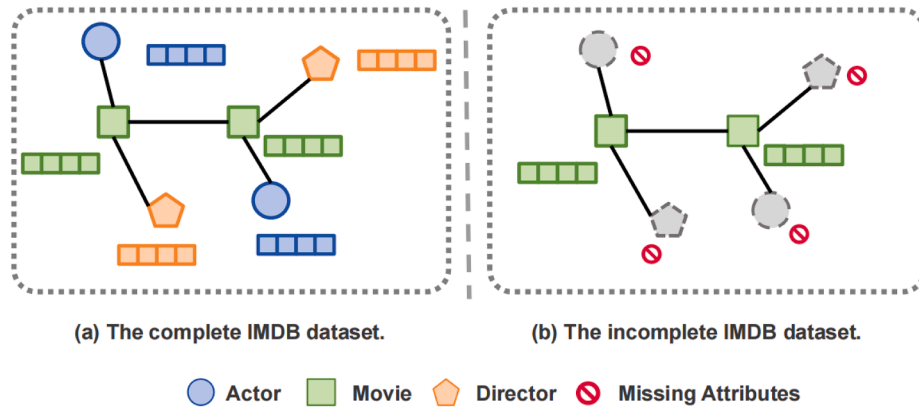


Fig. 1. Comparison of heterogeneous graphs with complete and incomplete attributes. (a) The IMDB dataset with complete attributes. (b) The IMDB dataset with incomplete attributes.

nodes. To complete the attributes, we utilize a context-aware attention mechanism to prevent non-neighbor information loss and fill in the missing node data. Furthermore, we directly employ improved graph attention networks for downstream tasks such as classification to improve computational efficiency. In summary, our contributions are as follows:

- We employ a collaborative random walk embedding approach that leverages meta-paths, enabling us to incorporate richer prior information into the node embedding process.
- We implement a context-aware attention mechanism for the purpose of attribute completion, which effectively prevents information loss between non-adjacent nodes.
- We utilize enhanced graph attention networks directly to perform downstream tasks to enhance model efficiency.
- The experiments conducted on three real-world datasets demonstrate that the EMC-Net exhibits better performance and can serve as a plug-and-play plugin for any baseline model. Our code is publicly available at <https://github.com/fyxxn/fy01.git>.

A preliminary version of this work has been submitted to a conference. The present work is a substantially extended version with three major improvements: (i) the graph neural network backbone has been upgraded from GAT to GATv2 to achieve more efficient downstream learning; (ii) the context-aware attention mechanism in the attribute completion module has been replaced with a self-attention mechanism to better capture global dependencies among node attributes; (iii) extensive new experiments, analyses, and discussions have been added to comprehensively evaluate the superiority and plug-and-play applicability of EMC-Net. Extensive experiments on three real-world datasets further verify the superiority and plug-and-play applicability of EMC-Net. The results, analyses, and discussions are new and not part of the conference submission.

This paper is organized as follows: [Section 2](#) reviews the related work and progress in heterogeneous graph attribute completion. [Section 3](#) provides the technical details of each component of the EMC-Net model. [Section 4](#) describes the datasets used in the experiments, the experimental setup, and presents the experimental results. Finally, [Section 5](#) summarizes our work.

2. Related work

2.1. Heterogeneous graph learning

The goal of heterogeneous graph learning is to embed the nodes and edges of a graph into a low-dimensional vector space while preserving both the structural and semantic information of the heterogeneous graph. Currently, there are two main types of methods for heterogeneous graph learning: shallow models and meta-path-based

models. Shallow models mainly adopt random walk methods for learning, such as DeepWalk (Perozzi et al., 2014), Metapath2vec (Dong et al., 2017), HIN2Vec (Fu et al., 2017), and HERec (Shi et al., 2018a), which utilize meta-path information to obtain better node embeddings. The aforementioned techniques have limitations as they only consider the graph's structural characteristics, neglecting the node attributes, which results in suboptimal outcomes. Meta-path-based models (Fu & King, 2022; Fu et al., 2020; Hu et al., 2020; Wang et al., 2019a) address this limitation by dividing the heterogeneous graph into multiple different homogeneous subgraphs and learning them separately using meta-path information. For example, ESIm (Shang et al., 2016) is a model based on user-defined meta-paths that guides the learning of node embeddings and is suitable for similarity search. Additionally, there are other heterogeneous graph learning methods. Chen et al. proposed the PME (Chen et al., 2018) model, which preserves the proximity of nodes by maintaining Euclidean distances, projecting different types of nodes into the same relational space for heterogeneous link prediction. To comprehensively describe the structure of heterogeneous graphs, Shi et al. introduced the HEER (Shi et al., 2018b) model, which embeds heterogeneous graphs via edge representations to capture the complex relationships between nodes and edges. Fan et al. proposed the metagraph2vec (Fan et al., 2018) model, which preserves the graph's structure and semantic information to the greatest extent, especially in tasks like malware detection. Sun et al. introduced a meta-graph-based network embedding model (Sun et al., 2018) that simultaneously considers the hidden relationships of all meta-information within a meta-graph, further enriching the semantic representations of the graph. In addition, graph learning methods have also been explored in application-oriented scenarios involving data heterogeneity, such as multi-modal brain graph learning for schizophrenia diagnosis (Huang et al., 2024) and federated connective brain template learning in a multi-domain non-IID setting (Chen et al., 2025).

2.2. Graph attribute completion

For dealing with common missing attributes in the real world, there are two types of methods: traditional machine learning methods and neural network-based methods. Traditional methods include mean interpolation (García-Laencina et al., 2010) and matrix factorization (Koren et al., 2009), while most neural network-based methods adopt an end-to-end network structure that learns both the completion of incomplete attributes and downstream tasks together, some of which use graph neural networks as the underlying infrastructure. These methods (Chen et al., 2020b; He et al., 2022; Jiang & Zhang, 2020; Jin et al., 2021) have shown good performance and potential, and thus, there are more and more works based on graph neural networks. The HGNNAC (Jin et al., 2021) method used a combination of attribute completion and

heterogeneous graph neural networks to handle heterogeneous graph data with missing attributes. In addition, methods, such as GCN (Kipf & Welling, 2017) and GAT (Veličković et al., 2017), have been extended to address the issue of incomplete attributes. These methods use attention mechanisms or graph convolutions to learn missing information. These models are especially effective when missing data is not randomly missing, but rather related to other attributes or the graph structure. Furthermore, methods, such as GraphSAGE (Hamilton et al., 2017), utilize neighbor aggregation strategies, aggregating features from neighboring nodes to handle missing node attributes and inferring missing values based on the local graph structure. These methods perform excellently in various graph tasks, such as node classification and link prediction, efficiently learning the underlying structure of the graph. Additionally, RA-HGNN (Zhao et al., 2024) proposes an effective solution by embedding different types of nodes into the same feature space. It uses topological embeddings to guide the completion of non-attribute nodes and incorporates a residual attention mechanism to handle attribute information between nodes.

2.3. Meta-path driven heterogeneous graph learning

Meta-paths are essential tools in heterogeneous graph analysis, capturing complex semantic relationships between different types of nodes. Sun et al. first introduced the concept of meta-paths in PathSim (Sun et al., 2011), a method that measures node similarity based on predefined sequences of node and edge types, demonstrating its effectiveness in the semantic analysis of heterogeneous information networks. Subsequent methods, such as MetaPath2Vec (Dong et al., 2017), utilize predefined meta-paths to guide random walks, generating semantically rich node sequences and applying the skip-gram (Mikolov et al., 2013) model to preserve both first-order and high-order similarities. This approach retains both structural and semantic information. Several variants based on this method have also been proposed. For instance, Spacey (He et al., 2019) introduces second-order supermatrices to unify different meta-paths and control transition probabilities between different node types; JUST (Hussein et al., 2018) employs jump-and-stay strategies to dynamically control transitions between node types; BHIN2Vec (Lee et al., 2019) frames heterogeneous graph embedding as a multi-task learning problem and balances the influence of different relationships by adjusting task weights; while HHNE (Wang et al., 2019b) performs meta-path-guided random walks in hyperbolic space (Helgason, 1979). Additionally, to address the limitation that reliance on predefined meta-paths causes the quality of results to depend heavily on the selected paths, Wang et al. proposed the Heterogeneous Graph Attention Network (HAN) (Zhang et al., 2019). This method dynamically learns the importance of meta-paths through node-level and semantic-level attention mechanisms, effectively modeling complex structures and rich semantics. However, the high computational complexity and dependence on careful parameter tuning limit the broader applicability of HAN.

2.4. Context-aware attention learning

The context-aware attention mechanism has shown exceptional performance in natural language processing and is gradually being applied to graph learning. Bahdanau et al. (2014) introduced the attention mechanism for aligning input and output sequences, laying the foundation for adaptive representation of sequential data. The Transformer (Vaswani et al., 2017) model improved upon this with self-attention, capturing dependencies in sequences, which led to breakthroughs in tasks like machine translation and text classification. In graph learning, GAT (Veličković et al., 2017) incorporated attention into graph neural networks, but it is limited to direct neighbors and often misses broader contextual information. HGT (Hu et al., 2020) extends attention mechanisms to heterogeneous graphs with context-aware attention, though its high computational demands limit its scalability. To address these, Wang et al. proposed HAN, which reduces computational

costs while maintaining effective representation learning through hierarchical attention. R-GAT (Wang et al., 2020) improves GAT by introducing attention for different relationship types, balancing efficiency and expressive power. Additionally, works, such as Graphormer (Ying et al., 2021), combine self-attention with graph-specific encoding (e.g., centrality, shortest paths), enhancing both local and global graph structure learning. Sparse Transformers (Child et al., 2019) use sparsification to reduce self-attention costs, enabling efficient learning on large-scale graph data. Cross-domain techniques like contrastive learning (Chen et al., 2020a) and knowledge distillation (Hinton, 2015) have also been integrated into attention mechanisms to improve robustness and generalization, helping capture more meaningful patterns in graph data.

3. Methodology

3.1. Problem formulation

Given a graph $G = (\mathcal{V}, \mathcal{E}, A, X)$ where $\mathcal{V} = \{v_0, v_1, \dots, v_N\}$ represents the set of N nodes, $\mathcal{E}$ represents the set of edges, $A \in \mathcal{R}^{N \times N}$ represents the adjacency matrix of the graph, and $X \in \mathcal{R}^{N \times d}$ represents the node feature matrix, where d is the dimension of node features.

Since the attributes of the graph are incomplete, the nodes can be divided into $\mathcal{V}_{cp}$ and $\mathcal{V}_{ms}$, where $\mathcal{V}_{cp}$ represents the nodes with complete attributes and $\mathcal{V}_{ms}$ represents the nodes with missing attributes. The feature matrix corresponding to $\mathcal{V}_{cp}$ is denoted as X_{cp} . Our goal is to complete the missing features for nodes $v \in \mathcal{V}_{ms}$ by combining the known node attributes and graph structural properties. We define X_{ac} as the completed node attributes, then $X_{new} = [X_{cp} | X_{ac}]$ is the new node feature matrix. Next, we input it into a graph neural network for downstream tasks.

3.2. Overall framework

Fig. 2 illustrates the proposed EMC-Net. Our model consists of three main components. Initially, we utilize the graph structure information to encode the nodes, followed by employing context-aware attention to fill in missing attributes. The completed attributes are subsequently fed into the heterogeneous graph neural network to facilitate downstream tasks. In the following sections, we will provide a detailed explanation of each module.

3.3. Meta-path-driven node embedding

The meta-path-driven random walk mainly involves computing the embedding of each node based on the graph structure, and its core idea can be summarized as “meta-path-constrained sampling with semantic encoding.” Meta-paths define valid node-type transition rules, guiding the random walk process to follow predefined semantic patterns rather than merely reflecting structural proximity. Given a meta-path $P : T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_l$, where T_i is the type of node v_i , the adoption probability of the i th step is defined as:

$$p(v_i | v_{i-1}, P) = \begin{cases} \frac{1}{|\mathcal{N}_{T_i}(v_{i-1})|}, & \text{if } A_{i-1,i} = 1 \text{ and } t(v_i) = T_i, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

where $|\mathcal{N}_{T_i}(v_{i-1})|$ represents the number of nodes in the neighborhood of node v_{i-1} that have a type equal to T_i , $t(v_i) = T_i$ is a mapping function that maps node v_i to its own type, and A denotes the adjacency matrix indicating whether two nodes are connected. The sampled node sequences are treated as semantic sequences, which are encoded into node embeddings through distributed representation learning (the skip-gram algorithm), ensuring that nodes appearing in similar semantic contexts are close to each other in the embedding space. This approach guarantees that node embeddings preserve both structural information and high-order semantic relations, providing a reliable representation space for semantic-driven attribute completion.

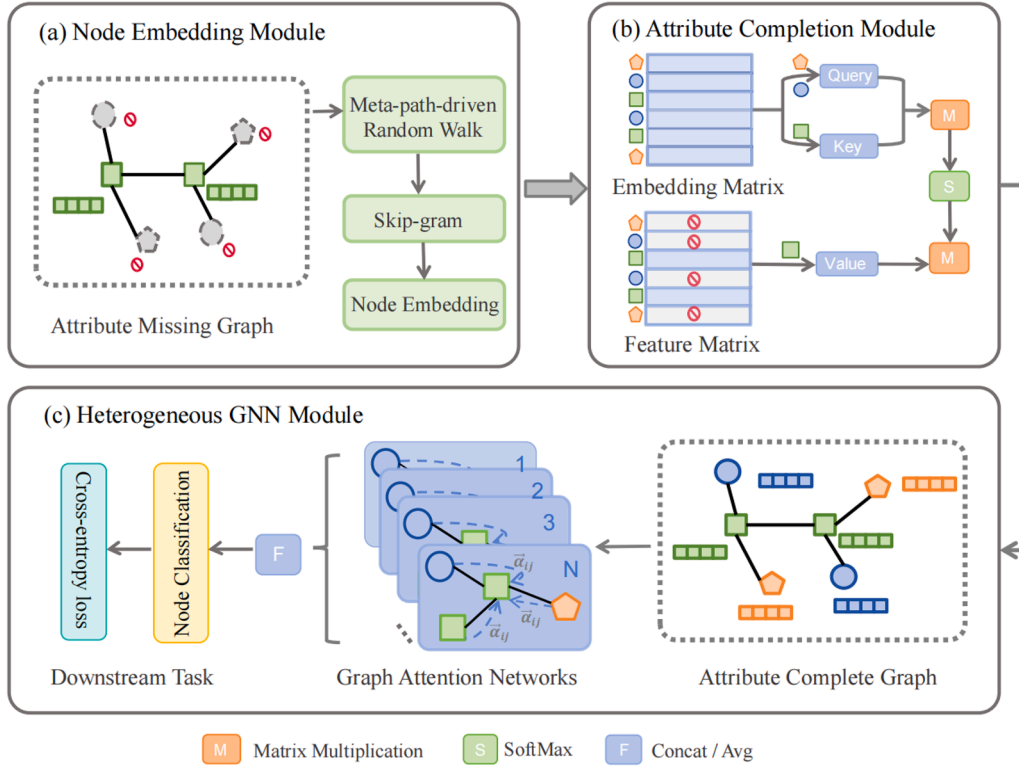


Fig. 2. The framework of EMC-Net is composed of three main modules: (a) Node Embedding Module. This module is designed to project nodes into a low-dimensional vector space while retaining their attribute information; (b) Attribute Completion Module. This module aims to recover the missing attributes by employing a context-aware attention mechanism and to model the correlations among attributes through the Attribute Complete Graph; (c) Heterogeneous GNN Module. This module focuses on processing heterogeneous graph data with graph attention networks, learning expressive node representations, and capturing the structural characteristics of the graph for various downstream tasks.

Then, the skip-gram algorithm is employed to encode the nodes of the graph. The skip-gram algorithm is a word vector representation learning algorithm used in natural language processing. Its goal is to learn the vector representation of each word, so that words with similar semantics are closer in vector space. The sampled node sequences can be viewed as sentences, and each node is a word in the sentence. Ultimately, for each node v_i , the embedding we obtain is represented as E_i , E_{cp} represents the embedding with complete attributes nodes, and E_{ms} represents the embedding with missing attributes nodes. The following pseudocode [Algorithm 1](#) outlines the steps for computing node embeddings.

Algorithm 1 Meta-path-driven node embedding.

```

1: Input: Graph  $G = (V, E)$ , Meta-path  $P$ 
2: Output: Node embeddings  $E$ 
3: Initialize node embeddings randomly
4: for each node  $v_i$  do
5:   Perform random walk for node  $v_i$ ;
6:   for each step in the random walk do
7:     Calculate next step based on meta-path  $P$ 
8:     Choose neighbor node based on meta-path constraints
9:   end for
10: end for
11: Use Skip-gram algorithm to update node embeddings
12: Return: Final node embeddings  $E$ 

```

3.4. Attribute completion module

In the attribute completion module, we mainly adopt an attention mechanism to complete missing attributes. The theoretical rationale for

this mechanism originates from the core requirement of heterogeneous graph attribute completion (HGAC): how to effectively leverage reliable semantic information from attribute-complete nodes while avoiding the limitations of relying solely on local neighborhoods. In heterogeneous graphs, semantic relationships between missing-attribute nodes and complete-attribute nodes are often propagated through non-local paths (e.g., high-order semantic relations captured by meta-paths). Traditional neighborhood aggregation-based methods implicitly assume that attribute dependencies are mostly limited to local topology, which fails to capture such non-local, semantic-driven dependencies.

Therefore, we embed missing-attribute nodes as Queries (Q), attribute-complete nodes as Keys (K), and features of attribute-complete nodes as Values (V), introducing a cross-attention mechanism to directly model the global semantic mapping between missing and complete nodes. This design theoretically aligns with the “semantic-consistency-based attribute completion” principle, ensuring that attribute inference relies on semantic similarity rather than mere topological proximity.

Moreover, meta-path-driven node embeddings have encoded structural and semantic priors, and the attention mechanism dynamically weights complete nodes that are semantically most relevant to missing nodes, avoiding indiscriminate aggregation and thus enhancing the stability and rationality of completion.

Attention mechanism is a commonly used technique in neural networks that combines different parts of information with different weights. In detail, the attention mechanism computes attention scores for each key-value pair in response to a query. It then combines the values using a weighted sum, where the weights are determined by the attention scores. The calculation formula for this process is as shown below:

$$\text{Attention}(Q, K, \bar{V}) = \text{softmax}(QK^T)\bar{V}, \quad (2)$$

where Q represents the query vector, K represents the key vector, $\bar{V}$ represents the value vector.

Additionally, inspired by the work of Transformer (Vaswani et al., 2017), we have incorporated the use of multi-head attention in our model. The mathematical representation for multi-head attention is as follows:

$$\text{MultiHead}(Q, K, \bar{V}) = \text{mean}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (3)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, \bar{V}W_i^V), \quad (4)$$

where W_i^Q , W_i^K , W_i^V , and W^O are learnable projection matrices. During attribute completion, our query is the embedding of the node with missing attributes E_{ms} , the key is the embedding of the node with complete attributes E_{cp} , and the value is the attributes of the node with complete attributes X_{cp} . Accordingly, the calculation formula for this process can be expressed as:

$$X_{ac} = \text{MultiHead}(E_{ms}, E_{cp}, X_{cp}), \quad (5)$$

$$X_{new} = [X_{cp} | X_{ac}]. \quad (6)$$

Before completing the attributes imputation, we initialize using one-hot encoding and learn the differences relative to the initialization. This helps accelerate the convergence speed. The pseudocode Algorithm 2 for the attribute completion process is as follows.

Algorithm 2 Attribute completion with context-aware attention.

```

1: Input: Node feature matrix  $X$ , adjacency matrix  $A$ 
2: Output: Completed node feature matrix  $X_{new}$ 
3: for each node  $v \in V_{ms}$  do
4:   Construct Query  $Q$ , Key  $K$ , and Value  $V$  for context-aware attention
5:   Compute multi-head attention:
6:   for each head  $h = 1, \dots, H$  do
7:      $\text{Attention}_h \leftarrow \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_k}}\right) V_h$ 
8:   end for
9:    $X_{completed}[v] \leftarrow \text{Concat}(\text{Attention}_1, \dots, \text{Attention}_H)W^O$ 
10:  Merge completed attributes with known attributes:
11:   $X_{new}[v] \leftarrow \text{Merge}(X_{completed}[v], X[v])$ 
12: end for
13: Return: Completed node feature matrix  $X_{new}$ 

```

3.5. Heterogeneous graph neural network

Upon acquisition of the attributes X_{new} , since the meta-path information of the heterogeneous graph has already been incorporated into the node embedding process, inspired by SimpleHGN (Lv et al., 2021), we have employed improved GAT for the purpose of graph node classification, thereby enhancing the model's computational efficiency.

The core idea of GAT (Veličković et al., 2017) is to calculate the attention weights between node pairs in the neighborhood of each node, and then use these weights as the weighted sum of the feature vectors of neighboring nodes to obtain a new representation for each node. Specifically, for node i and its neighbor node j , GAT calculates the attention weight e_{ij} between them and normalizes it to obtain w_{ij} . Then, for node i , GAT weights the feature vectors x_j of all its neighbor nodes with w_{ij} and sums them up to obtain the new feature vector x_i . More complex graph neural networks can be constructed by stacking multiple GAT layers. The formula for the traditional GAT is as follows:

$$x_i^{l+1} = \sigma \left(\sum_{j \in \mathcal{N}_i} w_{ij}^l W^l x_j^l \right), \quad (7)$$

where x_i^{l+1} represents the feature vector of node i in the $l+1$ layer, x_j^l represents the feature vector of node j in the l layer, $\mathcal{N}_i$ is the set of

neighboring nodes of node i , W^l is a learnable weight matrix, and σ is an activation function.

Inspired by SimpleHGN (Lv et al., 2021), we make some improvements to GAT to handle heterogeneous graphs. Next, we will provide a detailed explanation. Firstly, we incorporate edge type information into the attention calculation. Specifically, the formula for our updated attention weight w_{ij} calculation is as follows:

$$w_{ij}^l = \frac{\exp \left(\text{LeakyReLU}(a^T [W x_i^l | W x_j^l | W_r r_{(i,j)}^l]) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left(\text{LeakyReLU}(a^T [W x_i^l | W x_k^l | W_r r_{(i,k)}^l]) \right)}, \quad (8)$$

where a is a learnable weight vector, LeakyReLU is a leaky rectified linear unit function, and $[\cdot | \cdot]$ denotes the concatenation operation, W_r represents a learnable transformation matrix, where $r_{(i,j)}^l$ denotes the feature of the edge between node i and node j in the l layer.

Furthermore, to mitigate the problem of excessive smoothing in a graph neural network, a potential solution is to incorporate residual connections. These connections are added between the node representations of different layers. Consequently, the node aggregation formula for the $l+1$ layer is adjusted as follows:

$$x_i^{l+1} = \sigma \left(\sum_{j \in \mathcal{N}_i} w_{ij}^l W^l x_j^l + W_{res}^{l+1} x_i^l \right), \quad (9)$$

where W_{res}^{l+1} represents a transformation matrix used in the residual connection, which is applied when the dimensions of the node representations in the l and $l+1$ layers are different.

In addition, we employ multi-head attention, as described in Section 3.4, and implemented $\mathcal{L}_2$ regularization on the output embedding, i.e.,

$$z_i = \frac{x_i^l}{\|x_i^l\|}, \quad (10)$$

where z_i is the output embedding of node i and x_i^l is the final representation from Eq. (9). The pseudocode Algorithm 3 describing the heterogeneous graph neural network process is as follows.

Algorithm 3 Heterogeneous graph neural network.

```

1: Input: Node feature matrix  $X_{new}$ , Adjacency matrix  $A$ , Edge type information
2: Output: Final node embeddings  $X_{new}$ 
3: Initialize node features  $X_{new}$ 
4: for each layer  $l$  do
5:   for each node  $i$  do
6:     Compute attention weight  $a_{ij}$  for each neighbor  $j$ 
7:     Update node  $i$ 's feature by the weighted sum of neighbors' features
8:   Apply residual connection to avoid over-smoothing
9:   end for
10:  Apply multi-head attention to optimize node embeddings
11:  Apply L2 regularization to node embeddings
12: end for
13: Return: Final node embeddings  $X_{new}$ 

```

4. Experiments

4.1. Datasets

In order to assess the effectiveness of our proposed framework, we opted to utilize three well-known heterogeneous graph datasets: ACM, IMDB, and DBLP. Additionally, we employed the same data partitioning approach as that employed in the SimpleHGN (Lv et al., 2021). A detailed description of each aforementioned dataset is provided below.

Table 1
Statistics of datasets.

Dataset	# Nodes	# Node Types	# Edges	# Edge Types	Target
DBLP	26,128	4	239,566	6	Author
ACM	10,942	4	547,872	8	Paper
IMDB	21,420	4	86,642	6	Movie

- **DBLP**: The DBLP dataset is a bibliography website for computer science. We use a commonly used subset for our analysis. It includes four types of nodes. The target node for our classification is the author, and there are four classes.
- **ACM**: The ACM dataset is a citation website for papers, and we extract a subset of it using the same principle as HAN (Wang et al., 2019a).
- **IMDB**: The IMDB dataset is collected from a movie website and includes three types of nodes. The target node for our classification is the movie.

We have designed two types of experiments to validate our method. The first type involves nodes with missing attributes that require classification. The representative dataset for this type is DBLP. The second type involves nodes with existing attributes that require classification, while attributes for other node types are missing. The representative datasets for this type are ACM and IMDB. For the node classification task, which is a supervised learning task, we used 24% of the nodes for training, 6% for validation, and 70% for testing for each dataset. The detailed data for the three datasets are summarized in Table 1.

Since the ACM, IMDB, and DBLP datasets inherently contain naturally missing attributes, we set a uniform missing rate for target nodes to ensure experimental fairness: 70% of the target nodes retain complete attributes, while 30% are configured with missing attributes. This rate is consistent with that adopted in HGNNAC and SimpleHGN. For non-target nodes, the natural missing rate of the original datasets is preserved. The positions and rates of missing attributes are kept consistent by fixing the random seed (seed = 42), thus avoiding the impact of random factors on experimental fairness.

4.2. Comparison methods

To validate the effectiveness of the proposed framework, we compare it with existing state-of-the-art methods. Adhering to the selection principle of “comprehensive coverage and strong representativeness”, we select representative models from three categories of methods, aiming to verify the necessity and advantages of the proposed method from the perspective of different modeling assumptions.

These three categories of methods are as follows: 1) Neural network methods for homogeneous graph processing, serving to highlight the limitations when ignoring heterogeneous structural and semantic information; 2) Mainstream neural network methods for heterogeneous graph processing, covering paradigms such as meta-path guidance, hierarchical attention, and heterogeneous aggregation, which are used to evaluate the effectiveness of existing modeling strategies in attribute-missing scenarios; 3) Methods combining attribute completion with graph representation learning, directly comparing the differences in theoretical assumptions and information utilization among different completion mechanisms. Collectively, these three categories of methods form a complete technical spectrum ranging from basic graph learning to heterogeneous optimization and task-specific modeling, ensuring the systematicness and fairness of the experimental comparisons.

Specifically, we selected GCN and GAT as the representative methods for homogeneous graph processing, HetGNN (Zhang et al., 2019), HAN (Wang et al., 2019a), MAGNN (Fu et al., 2020), and SimpleHGN (Lv et al., 2021) as the representative methods for heterogeneous graph processing, and HGNNAC (Jin et al., 2021) as the representative method for combining attribute completion with graph representation

learning. Some of the baseline experimental results come from Zhu et al. (2023). These comparisons help us better understand the current neural network technologies for processing graph-structured data and evaluating the effectiveness of our proposed framework. The following are the introductions to each method:

- **HetGNN** is an innovative model for heterogeneous graph representation learning, designed to simultaneously capture the heterogeneity of graph structure and node content. HetGNN employs a restart-based random walk strategy to sample heterogeneous neighbors and groups them by node types, ensuring neighbor diversity. This design consists of two modules: the first module utilizes Recurrent Neural Networks (RNN) to encode node content features and generate content embeddings; the second module leverages an attention mechanism to aggregate content embeddings from different types of neighbors, thereby obtaining the final node representation.
- **HAN** is an innovative heterogeneous graph neural network model that captures both node-level and semantic-level heterogeneity through a hierarchical attention mechanism. HAN first maps features of different node types into a shared space, then dynamically learns the importance between nodes and their meta-path neighbors through node-level and semantic-level attention mechanisms, optimizing meta-path selection, and effectively aggregating neighbor features.
- **MAGNN** is an innovative heterogeneous graph embedding model designed to effectively capture both structural and semantic information in graphs. MAGNN overcomes the limitations of missing node attributes, ignoring intermediate nodes, and considering a single meta-path by employing node content transformation, intra-meta-path aggregation, and inter-meta-path aggregation. Specifically, MAGNN maps heterogeneous node features into a unified latent space and uses attention-based intra-meta-path aggregation to integrate neighbor information and capture complex relationships. Ultimately, MAGNN fuses information from multiple meta-paths through inter-meta-path aggregation to generate node embeddings.
- **Simple-HGN** is an extremely simple heterogeneous graph neural network model based on GAT, enhanced by three techniques: learnable type embeddings, residual connections, and L2 normalization. These techniques improve the performance of heterogeneous graph learning by leveraging type information, enhancing model expressiveness, and stabilizing the representations.
- **HGNN-AC** is an innovative framework designed to address the problem of missing node attributes in heterogeneous graphs. Unlike traditional methods, HGNN-AC uses a learnable attribute completion mechanism, leveraging node topological relationships to automatically complete missing attributes for nodes without attributes, and integrates this with GNN models to enhance overall performance. There are fundamental differences in the core mechanisms between HGNN-AC and the proposed EMC-Net: 1) HGNN-AC only propagates features based on local neighborhood topology, while EMC-Net captures high-order semantic similarity via meta-path-driven embedding, breaking the limitation of direct neighborhoods. 2) HGNN-AC adopts static neighborhood aggregation to integrate neighborhood information undifferentiatedly; in contrast, EMC-Net introduces a cross-attention mechanism to dynamically assign weights, selectively fusing information from nodes with complete attributes to avoid irrelevant interference. 3) HGNN-AC adopts a two-stage “completion-before-training” paradigm, where attribute completion is decoupled from downstream tasks; however, EMC-Net integrates the two into a unified framework, enabling end-to-end optimization and improving modeling efficiency and performance.

4.3. Implementation details

During training, all models were optimized using the Adam optimizer, and all experiments were conducted on an NVIDIA A100 GPU.

For the main experiments, we used a learning rate of 1e-3 and a weight decay of 1e-4 for the ACM and DBLP datasets, training for 500 epochs. For the IMDB dataset, the learning rate was set to 1e-3 with a weight decay of 2e-4, and the training process was conducted for 300 epochs. The negative slope parameter of LeakyReLU was fixed at 0.05. The software environment included Python 3.8.18, PyTorch 1.8.1 + cu101, torchvision 0.9.1 + cu101, torchaudio 0.8.1, numpy 1.19.5, scipy 1.5.4, scikit-learn 0.24.2, and DGL 0.6.1-cu101.

For the first ablation study, we adopted different training configurations: DBLP was trained with a learning rate of 2e-3 and a weight decay of 2e-5 for 500 epochs; ACM with a learning rate of 1e-3 and a weight decay of 1e-4 for 500 epochs; and IMDB with a learning rate of 5e-4, a weight decay of 2e-4, and a hidden layer size of 128. For the second ablation study, we used a learning rate of 1e-3 and a weight decay of 1e-4 for DBLP and ACM datasets, training for 500 epochs, while for IMDB we set the learning rate to 5e-4, the weight decay to 5e-4, and the negative slope of LeakyReLU to 0.2, with 500 training epochs.

4.4. Evaluation metrics

In this study, we utilized Macro-F1 (11) and Micro-F1 (12) as evaluation metrics to comprehensively assess the performance of our proposed framework in the node classification task. Specifically, Macro-F1 calculates the F1 score for each class and averages them, effectively reflecting the model's performance across different classes, especially in scenarios with class imbalance. In contrast, Micro-F1 aggregates the contributions of all classes to compute a global F1 score, thereby emphasizing the performance on the majority classes.

$$\text{Macro-F1} = \frac{1}{C} \sum_{i=1}^C \text{F1}_i, \quad (11)$$

where C denotes the total number of classes in the dataset, and F1_i represents the F1 score of class i .

$$\text{Micro-F1} = \frac{2 \cdot \sum_{i=1}^C TP_i}{2 \cdot \sum_{i=1}^C TP_i + \sum_{i=1}^C FP_i + \sum_{i=1}^C FN_i}, \quad (12)$$

where TP_i , FP_i , and FN_i denote the true positives, false positives, and false negatives of class i , respectively.

In our study, the DBLP dataset, representing the task of node classification with missing attributes, is evaluated with Macro-F1 to measure the model's accuracy when target nodes lack original attributes. On the other hand, Micro-F1 computes the weighted average F1 score across all classes, providing a better reflection of the model's overall performance. In datasets like ACM and IMDB, where target nodes already have attributes, Micro-F1 still evaluates the improvement in overall classification performance by completing the attributes of non-target nodes.

The combination of these two evaluation metrics allows for a comprehensive assessment of the effectiveness and stability of our EMC-Net framework in both tasks of node classification with missing attributes and with known attributes. Through comparisons with existing state-of-the-art methods, our framework demonstrates significant performance improvements across multiple datasets, further validating its advantages in node classification tasks.

4.5. Experimental results

Table 2 presents a performance evaluation of EMC-Net in comparison to other graph neural network techniques. The study initially compared EMC-Net with the homogeneous neural network, including GCN and GAT, and observed a significant improvement of 0.79%–14.31% across three datasets. Furthermore, when compared to heterogeneous neural networks, such as HetGNN, HAN, MAGNN, and SimpleHGN, EMC-Net demonstrated superior performance with an improvement of 0.52%–3.37% compared to the best-performing method, SimpleHGN. This indicates the effectiveness of the completion module we added

to our method. When comparing our method with HGNNAC, which only considers neighbor completion, we found that our context-aware attention completion significantly improves stability and effectiveness, achieving at least 1.82%–14.96% improvement on all three datasets. All of them effectively demonstrate the effectiveness of our EMC-Net.

Additionally, by observing the performance of the DBLP dataset, we found that our EMC-Net can significantly improve node classification when the target nodes lack original attributes. This indicates that our model can effectively learn the features of known attribute nodes and graph structure features to make correct classifications. Furthermore, for datasets such as ACM and IMDB that already have target node attributes, our method can still improve performance by completing non-target nodes.

To provide a more intuitive comparison between different models, we conducted a visualization task on the DBLP dataset, learning node embeddings for MAGNN, GAT, MAGNN + EMC-Net, and EMC-Net. MAGNN + EMC-Net refers to applying our proposed completion method to the MAGNN baseline network. We used t-SNE (Van der Maaten & Hinton, 2008) to project the embeddings into a two-dimensional space and colored the nodes based on their class. The results are shown in Fig. 3.

From the visualizations, it can be observed that GAT performed the worst, and some authors belonging to different classes had blurred boundaries. MAGNN performed relatively better than GAT, but the authors of the same class were scattered and lacked sufficient clustering. When MAGNN was combined with our proposed framework, MAGNN + EMC-Net exhibited denser cluster structures, enabling better differentiation of different classes in the visualizations. Our EMC-Net produced clearly defined boundaries and dense cluster structures after clustering, indicating that our model significantly improved the performance of the clustering task and could effectively differentiate authors from different research fields.

Further analysis reveals that the root cause of the clustering differences lies in the varying abilities of different models to capture semantic and structural information: As a homogeneous model, GAT ignores node type differences and meta-path semantics, leading to mixed clustering of authors from different fields with blurred boundaries; As a heterogeneous model, MAGNN incorporates meta-path aggregation but lacks attribute completion, resulting in incomplete semantic representations of nodes with missing attributes—thus, authors of the same class are scattered and the clustering effect is insufficient; By integrating our completion method, MAGNN + EMC-Net improves the clustering effect of nodes with missing attributes, but its clustering density is still lower than that of EMC-Net due to the limitations of MAGNN's own aggregation strategy; EMC-Net encodes semantic structures through meta-path embedding, completes missing attributes via context-aware attention, and enhances aggregation performance using an improved GAT. Ultimately, it forms cluster structures with clear boundaries and dense intra-class distributions, demonstrating that the model can accurately capture the semantic features of authors from different research fields and achieve effective class distinction.

4.6. Ablation studies

In order to verify the effectiveness of each module, we conducted the following ablation experiments.

Effectiveness of Network Design. We divided the model into three modules: initialization module (INIT), completion module (AC), and heterogeneous graph network module (HIN). Then we removed these three modules separately to observe their impact on performance. “-AC” represents the use of zero imputation. “-HIN” represents the use of MAGNN instead of improved GAT for the heterogeneous graph network module. “-INIT” represents the non-use of one-hot initialization. The experimental results are presented in Table 3. From the experimental results, it can be seen that among these three modules, INIT has the greatest impact.

Table 2

Quantitative comparison of EMC-Net and state-of-the-art methods on three benchmark datasets. The best results are in **boldface**.

Model	DBLP		ACM		IMDB	
	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
GCN	90.54±0.27	91.18±0.25	92.63±0.23	92.60±0.22	59.95±0.72	65.35±0.35
GAT	92.96±0.35	93.46±0.35	92.41±0.84	92.39±0.84	56.95±1.55	64.24±0.55
HetGNN	92.77±0.24	93.23±0.23	84.93±0.78	84.83±0.76	47.87±0.33	50.83±0.26
HAN	93.17±0.19	93.64±0.17	87.68±1.94	87.73±1.81	59.70±0.90	65.61±0.54
MAGNN	93.16±0.38	93.65±0.34	91.06±1.44	90.95±1.43	56.92±1.76	65.11±0.59
SimpleHGN	93.83±0.18	94.25±0.19	92.92±0.67	92.85±0.68	62.98±1.66	67.42±0.42
HGNNAC	92.97±0.72	93.43±0.69	90.89±0.87	90.83±0.87	56.63±0.81	63.85±0.85
EMC-Net	94.74±0.13	95.13±0.12	93.42±0.36	93.33±0.38	65.10±0.30	68.21±0.31

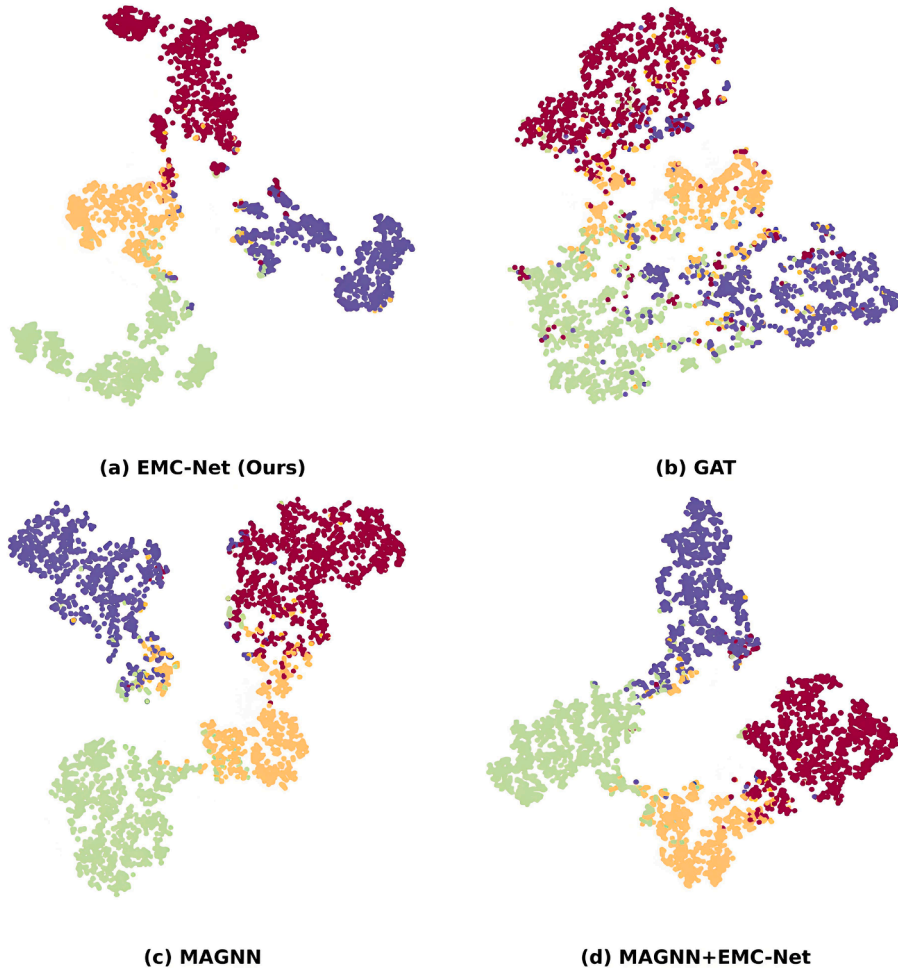


Fig. 3. Visualization of the embeddings on DBLP. Each point represents an author, with colors indicating research fields.

Table 3

Effectiveness of network design. The effectiveness of our components is demonstrated by removing different components from our model.

Model	DBLP		ACM		IMDB	
	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
-AC	93.83±0.18	94.25±0.19	92.92±0.67	92.85±0.68	62.98±1.66	67.42±0.42
-HIN	93.53±0.31	94.00±0.31	92.14±0.92	92.04±0.95	58.00±1.10	65.53±1.10
-INIT	81.00±0.23	81.52±0.24	92.17±0.84	92.10±0.83	53.25±1.26	61.30±0.51
EMC-Net	94.74±0.13	95.13±0.12	93.42±0.36	93.33±0.38	65.10±0.30	68.21±0.31

Table 4

Attribute completion module effectiveness analysis. Compare our completion method with other completion methods.

Model	DBLP		ACM		IMDB	
	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
ZERO-AC	93.83±0.18	94.25±0.19	92.92±0.67	92.85±0.68	62.98±1.66	67.42±0.42
ONE-HOT	93.50±0.56	94.00±0.53	93.15±0.73	93.06±0.38	63.72±0.72	67.52±0.32
HGNNAC	93.24±0.49	93.73±0.45	93.16±0.24	93.09±0.23	64.44±1.13	67.67±0.39
EMC-Net	94.74±0.13	95.13±0.12	93.42±0.36	93.33±0.38	65.10±0.30	68.21±0.31

Table 5

Validation of model plug-and-play capability. Applying our model to different heterogeneous graph neural network benchmark models.

Model	DBLP		ACM		IMDB	
	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
MAGNN	93.16±0.38	93.65±0.34	91.06±1.44	90.95±1.43	56.92±1.76	65.11±0.59
MAGNN-HGNNAC	92.97±0.72	93.43±0.69	90.89±0.87	90.83±0.87	56.63±0.81	63.85±0.85
MAGNN-Ours	93.53±0.31	94.00±0.31	92.14±0.92	92.44±0.95	58.00±1.10	65.53±1.10
SimpleHGN	93.83±0.18	94.25±0.19	92.92±0.67	92.85±0.68	62.98±1.66	67.42±0.42
SimpleHGN-HGNNAC	93.24±0.49	93.73±0.45	93.16±0.24	93.09±0.23	64.44±1.13	67.67±0.39
SimpleHGN-Ours	94.74±0.13	95.13±0.12	93.42±0.36	93.33±0.38	65.10±0.30	68.21±0.31

Effectiveness of Attribute Completion Module. We conducted ablation studies on different completion methods, “ZERO-AC” refers to completion using zero filling, “ONE-HOT” refers to completion using one-hot encoding, “HGNNAC” refers to completion using the method described in the HGNNAC (Jin et al., 2021). The experimental results are presented in Table 4. It is evident that our completion method surpasses the performance of the three preceding completion methods across all three datasets, thereby showcasing the efficacy and reliability of our completion approach.

Validation of Model Plug-and-play Capability. We applied our proposed completion method and the completion method from the HGNNAC to MAGNN and SimpleHGN. Table 5 results demonstrate that our completion method has a significant improvement on both MAGNN and SimpleHGN. Moreover, the experimental results also indicate that our completion module can be used as a plug-and-play method for any graph neural network that deals with graph representation learning, which is one of the important advantages of our method.

The *plug-and-play* property of EMC-Net originates from its modular design principle. EMC-Net consists of a node embedding module and an attribute completion module, producing a complete node feature matrix X_{new} , which can be directly used as input to any heterogeneous graph neural network (e.g., MAGNN, SimpleHGN) without modifying the core architecture of these baseline models. When integrating EMC-Net into MAGNN or SimpleHGN, one only needs to replace the original feature input of the baseline model with X_{new} , while all other components, such as aggregation layers and classification layers, remain unchanged. Experimental results show that, compared with the original MAGNN, MAGNN + EMC-Net achieves an improvement of 0.37%–1.08% in Macro-F1 and 0.35%–1.49% in Micro-F1. Similarly, compared with the original SimpleHGN, SimpleHGN + EMC-Net achieves an improvement of 0.50%–2.12% in Macro-F1 and 0.50%–0.88% in Micro-F1. These results demonstrate that EMC-Net can be embedded as an independent module into existing heterogeneous graph models without reconstructing the baseline models, fully satisfying the definition of a *plug-and-play* module.

Validation of Computational Efficiency. To validate the efficiency of our model, we created an ablated version that employs a heterogeneous graph neural network for downstream tasks after attribute completion, as in Lv et al. (2021). The results show that our EMC-Net is about six times faster than the ablated version, demonstrating its high efficiency.

We conducted a runtime comparison experiment between EMC-Net and baseline models, MAGNN and SimpleHGN, on the ACM dataset us-

ing an NVIDIA A100 GPU with 500 training epochs and five repeated runs. MAGNN and SimpleHGN represent the first and second best performing baseline methods in terms of overall performance, respectively. The experimental results show that the average training time of MAGNN is 125.28 s and that of SimpleHGN is 131.91 s, while EMC-Net requires only 37.51 s. These results indicate that EMC-Net achieves substantially lower computational cost than the two strongest baseline methods under the same experimental setting.

Experimental Evaluation of the Enhanced Attribute Completion Module. In this experiment, we replaced the attention mechanism (context-aware attention) in the attribute completion module with a self-attention mechanism to better capture the complex relationships within the data. There are significant differences in the applicable scenarios between context-aware attention and self-attention: context-aware attention is more suitable for scenarios where partial node attributes are missing, and its cross-set design avoids interference from invalid information of missing-attribute nodes, thereby preventing semantic drift; in contrast, self-attention is more applicable to scenarios with a low attribute missing rate or high-quality node embeddings. Under such circumstances, global interactions can fully explore the potential correlations between nodes without being restricted by meta-path predefinition. Thus, when self-attention meets specific prerequisites (e.g., high embedding quality and moderate missing degree), it can enhance the attribute completion effect through more sufficient global interactions.

It should be clarified that there are fundamental differences between the proposed context-aware attention and self-attention: 1) The former is “cross-set interaction” (precise matching between the set of nodes with missing attributes and the set of nodes with complete attributes), while the latter is “intra-set interaction” (capturing global node correlations); 2) The Q/K/V of the former derive from nodes with missing attributes and nodes with complete attributes (ensuring the reliability of completed information), whereas the Q/K/V of the latter cover all nodes (balancing global interactions).

Their innovations are: 1) The context-aware attention binds meta-path embedding to cross-set interaction, and adopts heterogeneous QKV (Q: nodes with missing attributes; K/V: nodes with complete attributes and their features) to transmit reliable information from semantically similar nodes. It can be modularly adapted to any heterogeneous graph network, optimizing semantic fragmentation and completion accuracy in HGAC tasks; 2) The self-attention takes meta-path embedding as input, globally mines potential semantic relationships, dynamically adjusts node weights to capture complex dependencies for improved

Table 6

The performance comparison before and after improving the attention mechanism in the attribute completion module.

Model	DBLP		ACM		IMDB	
EMC-Net	94.74±0.13	95.13±0.12	93.42±0.36	93.33±0.38	65.10±0.30	68.21±0.31
EMC-Net(AM)	94.86±0.34	95.28±0.31	93.82±0.17	93.73±0.19	65.55±0.34	68.43±0.33

Table 7

The performance comparison of the backbone network before and after improvement.

Model	DBLP		ACM		IMDB	
	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
EMC-Net	94.74±0.13	95.13±0.12	93.42±0.36	93.33±0.38	65.10±0.30	68.21±0.31
EMC-Net(GATv2)	95.11±0.19	95.49±0.18	94.10±0.22	94.03±0.19	65.52±0.55	68.23±0.49

attribute representation, and focuses on high-information regions to reduce redundancy and accelerate convergence.

Table 6 presents a performance comparison before and after incorporating the self-attention mechanism. The model with self-attention outperformed the original EMC-Net across all three datasets, with improvements ranging from 0.13% to 0.69%. These results demonstrate the effectiveness of self-attention in improving feature representation, learning efficiency, and prediction accuracy.

In addition to improved accuracy, the model with self-attention also exhibited enhanced learning efficiency. The mechanism facilitated faster convergence during training by focusing computational resources on the most informative graph regions, thus reducing redundancy. This efficiency not only accelerates the training process but also makes the model more scalable to larger datasets and real-world applications.

Effect of Backbone Network Improvement (GATv2). To enhance EMC-Net, we replaced the original backbone with GATv2. GATv2 improves the traditional GAT by introducing a more flexible, learnable attention mechanism that better captures higher-order dependencies and complex relationships in the graph. Unlike GAT, which uses fixed attention weights based on immediate neighbors, GATv2 dynamically adjusts attention weights based on node features and context, resulting in improved node representations and more accurate predictions. GATv2 addresses the limitations of GAT, providing refined attention mechanisms, better information aggregation across distant nodes, and more adaptive representations for diverse nodes. This makes it ideal for tasks like node classification and link prediction, which require capturing complex dependencies.

Table 7 shows that the EMC-Net variant employing GATv2 as the backbone outperformed the original EMC-Net across all datasets, with performance improvements ranging from 0.39% to 0.75%. This demonstrates the effectiveness of the GATv2 backbone in enhancing feature extraction, representation learning, and downstream task performance, emphasizing the importance of advanced backbone structures in deep learning models.

In addition, the improvements in the GATv2 backbone have significantly enhanced training efficiency and the model's generalization capability. Compared to traditional methods, it demonstrates greater robustness when handling large-scale data and exhibits higher resilience to noise and incomplete attributes. These advantages make GATv2 an ideal choice for building advanced graph neural networks.

5. Conclusion

In this work, we have proposed an effective model, EMC-Net, to address the issue of attributes missing in heterogeneous graphs. Our EMC-Net adopts a collaborative random walk embedding scheme based on meta-path, which incorporates prior knowledge of meta-path during node sampling. During the attribute completion process, it utilizes a context-aware attention mechanism to complete the information of nodes with missing attributes and avoid information loss. Finally, it

directly employs improved graph attention networks for downstream tasks such as classification, effectively improving computational efficiency. Furthermore, we made two key improvements to the model. First, we replaced the original attention mechanism in the attribute completion module with self-attention, allowing the model to dynamically adjust node importance based on interactions, thereby capturing complex dependencies and improving attribute representation. Second, we upgraded the backbone network by using GATv2, enabling better capture of higher-order dependencies and adaptive attention weights, resulting in more accurate node representations and improved task performance. Extensive experiments on three real-world heterogeneous graph datasets demonstrate that EMC-Net is effective in HGAC and outperforms state-of-the-art methods.

However, the current method still has several limitations: 1) Meta-paths rely on manual predefinition, which may affect embedding quality if improperly selected; 2) The computational complexity of context-aware attention is $O(N \times M)$ (N is the number of missing nodes, M is the number of complete nodes), which may pose efficiency challenges on large-scale heterogeneous graphs; 3) Current work focuses on node attribute completion and has not extended to edge attribute completion. Future research directions include: 1) Exploring automatic meta-path learning methods to reduce manual dependence; 2) Optimizing sparse attention mechanisms (e.g., Top-K semantic matching) to reduce computational costs on large-scale graphs; 3) Extending to edge attribute completion using edge-type information along meta-paths to achieve joint node-edge attribute completion.

CRedit authorship contribution statement

Geng Chen: Writing – review & editing, Validation, Supervision, Funding acquisition, Formal analysis, Conceptualization; **Yuan Feng:** Writing – original draft, Visualization, Validation, Methodology, Investigation, Data curation, Conceptualization; **Lijun Zhang:** Validation, Supervision, Methodology, Data curation; **Xiaoyu Bai:** Writing – review & editing, Validation; **Qingyue Wang:** Writing – review & editing; **Peng Wang:** Writing – review & editing, Supervision, Project administration, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the [National Natural Science Foundation of China](#) (Nos. 62571451, U19B2037), the Innovation Capability Support Program of Shaanxi (No. 2025ZC-KJXX-47), the Practice and Innovation Funds for Graduate Students of Northwestern Polytechnical

University (No. PF2024012), the National Key R&D Program of China (No. 2020AAA), the Shaanxi Provincial Key R&D Program (No. 2021KWZ-03), and the Natural Science Basic Research Program of Shaanxi (No. 2021JCW-03).

References

- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Chen, G., Wang, Q., Feng, Y., & Rekik, I. (2025). Metadata-driven federated learning of connective brain templates in non-IID multi-domain scenarios. *IEEE Transactions on Medical Imaging*.
- Chen, H., Yin, H., Wang, W., Wang, H., Nguyen, Q. V. H., & Li, X. (2018). PME: Projected metric embedding on heterogeneous networks for link prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1177–1186).
- Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020a). A simple framework for contrastive learning of visual representations. In *International conference on machine learning* (pp. 1597–1607). PMLR.
- Chen, X., Chen, S., Yao, J., Zheng, H., Zhang, Y., & Tsang, I. W. (2020b). Learning on attribute-missing graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44 (2), 740–757.
- Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*.
- Dong, Y., Chawla, N. V., & Swami, A. (2017). metapath2vec: Scalable representation learning for heterogeneous networks. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 135–144).
- Fan, Y., Hou, S., Zhang, Y., Ye, Y., & Abdulhayoglu, M. (2018). Gotcha-sly malware! Scorpion a metagraph2vec based malware detection system. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 253–262).
- Fu, T.-y., Lee, W.-C., & Lei, Z. (2017). Hin2vec: Explore meta-paths in heterogeneous information networks for representation learning. In *Proceedings of the 2017 ACM on conference on information and knowledge management* (pp. 1797–1806).
- Fu, X., & King, I. (2022). Mecch: Metapath context convolution-based heterogeneous graph neural networks. *arXiv preprint arXiv:2211.12792*.
- Fu, X., Zhang, J., Meng, Z., & King, I. (2020). MaGNN: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the web conference 2020* (pp. 2331–2341).
- García-Laencina, P. J., Sancho-Gómez, J.-L., & Figueiras-Vidal, A. R. (2010). Pattern classification with missing data: A review. *Neural Computing and Applications*, 19, 263–282.
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30, 1024–1034.
- He, D., Liang, C., Huo, C., Feng, Z., Jin, D., Yang, L., & Zhang, W. (2022). Analyzing heterogeneous networks with missing attributes by unsupervised contrastive learning. *IEEE Transactions on Neural Networks and Learning Systems*, 35 (4), 4438–4450.
- He, Y., Song, Y., Li, J., Ji, C., Peng, J., & Peng, H. (2019). Hetespacewalk: A heterogeneous spacey random walk for heterogeneous information network embedding. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 639–648).
- Helgason, S. (1979). Differential geometry, lie groups, and symmetric spaces. Academic Press.
- Hinton, G. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Hu, Z., Dong, Y., Wang, K., & Sun, Y. (2020). Heterogeneous graph transformer. In *Proceedings of the web conference 2020* (pp. 2704–2710).
- Huang, Y., Chen, G., Wen, X., & Shen, D. (2024). Multi-modal brain graph learning of shared-specific features for schizophrenia diagnosis. In *2024 IEEE International conference on bioinformatics and biomedicine (BIBM)* (pp. 1522–1525). IEEE.
- Hussein, R., Yang, D., & Cudré-Mauroux, P. (2018). Are meta-paths necessary? Revisiting heterogeneous graph embeddings. In *Proceedings of the 27th ACM international conference on information and knowledge management* (pp. 437–446).
- Jiang, B., & Zhang, Z. (2020). Incomplete graph representation and learning via partial graph neural networks. *arXiv preprint arXiv:2003.10130*.
- Jin, D., Huo, C., Liang, C., & Yang, L. (2021). Heterogeneous graph neural network via attribute completion. In *Proceedings of the web conference 2021* (pp. 391–400).
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations*.
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42 (8), 30–37.
- Lee, S., Park, C., & Yu, H. (2019). Bhin2vec: Balancing the type of relation in heterogeneous information network. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 619–628).
- Lv, Q., Ding, M., Liu, Q., Chen, Y., Feng, W., He, S., Zhou, C., Jiang, J., Dong, Y., & Tang, J. (2021). Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 1150–1160).
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 3111–3119.
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 701–710).
- Rubin, D. B. (2004). Multiple imputation for nonresponse in surveys (vol. 81). John Wiley & Sons.
- Shang, J., Qu, M., Liu, J., Kaplan, L. M., Han, J., & Peng, J. (2016). Meta-path guided embedding for similarity search in large-scale heterogeneous information networks. *arXiv preprint arXiv:1610.09769*.
- Shi, C., Hu, B., Zhao, W. X., & Philip, S. Y. (2018a). Heterogeneous information network embedding for recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 31 (2), 357–370.
- Shi, Y., Zhu, Q., Guo, F., Zhang, C., & Han, J. (2018b). Easing embedding learning by comprehensive transcription of heterogeneous information networks. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2190–2199).
- Sun, L., He, L., Huang, Z., Cao, B., Xia, C., Wei, X., & Philip, S. Y. (2018). Joint embedding of meta-path and meta-graph for heterogeneous information networks. In *2018 IEEE International conference on big knowledge (ICBK)* (pp. 131–138). IEEE.
- Sun, Y., Han, J., Yan, X., Yu, P. S., & Wu, T. (2011). Paths: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, 4(11), 992–1003.
- Taguchi, H., Liu, X., & Murata, T. (2021). Graph convolutional networks for graphs containing missing features. *Future Generation Computer Systems*, 117, 155–168.
- Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9 (11), 2579–2605.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2017). Graph attention networks. In *International conference on learning representations*.
- Wang, K., Shen, W., Yang, Y., Quan, X., & Wang, R. (2020). Relational graph attention network for aspect-based sentiment analysis. *arXiv preprint arXiv:2004.12362*.
- Wang, X., Ji, H., Shi, C., Wang, B., Ye, Y., Cui, P., & Yu, P. S. (2019a). Heterogeneous graph attention network. In *The world wide web conference* (pp. 2022–2032).
- Wang, X., Zhang, Y., & Shi, C. (2019b). Hyperbolic heterogeneous information network embedding. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 5337–5344). (vol. 33).
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., & Liu, T.-Y. (2021). Do transformers really perform badly for graph representation? *Advances in Neural Information Processing Systems*, 34, 28877–28888.
- Zhang, C., Song, D., Huang, C., Swami, A., & Chawla, N. V. (2019). Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 793–803).
- Zhao, Z., Liu, Z., Wang, Y., Yang, D., & Che, W. (2024). RA-HGNN: Attribute completion of heterogeneous graph neural networks based on residual attention mechanism. *Expert Systems with Applications*, 243, 122945.
- Zhu, G., Zhu, Z., Wang, W., Xu, Z., Yuan, C., & Huang, Y. (2023). AutoAC: Towards automated attribute completion for heterogeneous graph neural network. *arXiv preprint arXiv:2301.03049*.